HPC Lab

Name: Aditi Mittal
Class: B.Tech CSE Final Year
PRN: 23610067
Batch: B1

Experiment no 5

To understand and implement task-level parallelism using OpenMP sections and task constructs, and to analyze their suitability and performance for applications containing independent, irregular, and dynamically generated tasks.

Why?

So far, we have used OpenMP mainly to parallelize **loops**. However, not every application can be naturally expressed as a parallel loop.

Consider the following situations:

- Two independent functions need to execute simultaneously.
- A program contains different operations with different execution times.
- The amount of work is not known in advance.
- An algorithm generates new work dynamically.
- A recursive algorithm creates smaller subproblems.
- Some computations can start only after other

computations finish. In such cases, task-level

parallelism becomes useful.

OpenMP provides:

- sections : suitable for a small, fixed number of independent tasks.
- task : suitable for dynamically generated or irregular tasks.

Task-based parallelism is used in several real-world applications.

Domain	Example
AI / Machine Learning	Parallel preprocessing and inference tasks
Computer Vision	Independent image-processing operations
Scientific Computing	Independent simulation components
Graph Processing	Irregular graph traversal

Domain	Example
Robotics	Parallel sensor-processing tasks
Multimedia	Audio/video processing pipelines
Algorithms	Recursive sorting and divide-and-conquer algorithms
Compilers	Parallel compilation of independent modules

Suppose an AI application has the following independent operations:

1. Image preprocessing
2. Feature extraction
3. Data normalization
4. Metadata processing

Would a parallel for be the most natural way to express this parallelism?

If the operations are different and have different execution times, task-level parallelism may be more appropriate.

Answer the following before starting the implementation.

1. What is the difference between data parallelism and task parallelism?
2. When is parallel for more suitable than task?
3. What is the purpose of the OpenMP sections construct?
4. What is the purpose of the OpenMP task construct?
5. Why can task creation introduce overhead?
6. Give two real-world applications where task-level parallelism may be preferable to loop parallelism.

1. Difference between data parallelism and task parallelism

Data parallelism applies the *same* operation to different pieces of data simultaneously (e.g. parallel for over array elements) — all threads run identical code. Task parallelism assigns *different* operations (potentially with different code, different execution times, and no shared loop structure) to different threads — e.g. one thread does image preprocessing while another does feature extraction.

2. When is parallel for more suitable than task?

parallel for is better when the work is a single, regular, statically-known-size loop where every iteration does the same kind of work — the compiler/runtime can divide iterations up front with minimal overhead. task is better when the work is irregular, of unpredictable size, generated dynamically (e.g. recursively), or consists of genuinely different operations rather than repetitions of one operation.

3. Purpose of the sections construct

sections lets you specify a small, fixed number of distinct code blocks (sections) inside a parallel sections region, and the runtime assigns each block to an available thread. It's designed for a known, small set of independent tasks — not for dynamically generated work.

4. Purpose of the task construct

task creates a unit of work that is queued and can be executed by any thread in the team, whenever a thread becomes free. Unlike sections, tasks can be created dynamically at runtime (including recursively, from inside other tasks), making them suitable for irregular or unpredictable workloads.

5. Why can task creation introduce overhead?

Every #pragma omp task requires the runtime to allocate bookkeeping for the task (capturing its data environment, e.g. via firstprivate/shared), place it in a task queue, and later schedule and dispatch it to a thread. If tasks are created for very small amounts of work, this bookkeeping cost can exceed the actual computation the task performs, making the parallel version slower than sequential.

6. Two real-world applications where task-level parallelism is preferable to loop parallelism

- Recursive divide-and-conquer algorithms (merge sort, quicksort, tree/graph traversal) — the amount and size of sub-work isn't known until runtime, which loop parallelism can't express.
- A multimedia/video processing pipeline where decoding, filtering, and encoding stages have different costs and data dependencies between them — this is a set of distinct operations, not repetitions of one operation over a data array.

Section A

Task 1: Independent Tasks using OpenMP Sections

Consider the following four independent operations:

1. Calculate the sum of an array.
2. Calculate the maximum element.
3. Calculate the minimum element.
4. Calculate the average of the array

Implement:

Part A – Sequential Version

Execute all four operations sequentially.

Part B – OpenMP Version

Use:
#pragma omp parallel sections
and divide the four operations into
separate sections. Each section should
perform one operation.

Requirements

1. Execute the program sequentially.
2. Execute the program using OpenMP sections.
3. Verify that both versions produce identical results.
4. Measure

execution time.

```c
/*
* Task 1: Sum / Max / Min / Average via OpenMP sections
* Compile: gcc -fopenmp Task1/array-op-sections.c -o array-op-sections
* Run:     ./array-op-sections <array_size> [threads]
*/
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
  int n = (argc > 1) ? atoi(argv[1]) : 1000000;
  int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
```

```
omp_set_num_threads(threads);


double *arr = malloc((size_t)n * sizeof(double));
srand(42);
for (int i = 0; i < n; i++) arr[i] = rand() % 1000;


/* ---------- Sequential ---------- */
double sum_seq = 0, max_seq = arr[0], min_seq = arr[0], avg_seq;
double t0 = omp_get_wtime();
for (int i = 0; i < n; i++) {
    sum_seq += arr[i];
    if (arr[i] > max_seq) max_seq = arr[i];
    if (arr[i] < min_seq) min_seq = arr[i];
}
avg_seq = sum_seq / n;
double t1 = omp_get_wtime();
double seq_time = t1 - t0;


/* ---------- OpenMP sections ---------- */
double sum_par = 0, max_par = arr[0], min_par = arr[0], avg_par;
double t2 = omp_get_wtime();
#pragma omp parallel sections
{
    #pragma omp section
    { double s = 0; for (int i = 0; i < n; i++) s += arr[i]; sum_par = s; }

    #pragma omp section
    { double m = arr[0]; for (int i = 0; i < n; i++) if (arr[i] > m) m = arr[i]; max_par = m; }

    #pragma omp section
```

```
    { double m = arr[0]; for (int i = 0; i < n; i++) if (arr[i] < m) m = arr[i]; min_par = m; }


    #pragma omp section

    { double s = 0; for (int i = 0; i < n; i++) s += arr[i]; avg_par = s / n; }

  }

  double t3 = omp_get_wtime();

  double par_time = t3 - t2;


  int correct = (sum_seq == sum_par) && (max_seq == max_par) &&

                (min_seq == min_par) && (avg_seq == avg_par);


  printf("Sequential time: %f s\n", seq_time);

  printf("Sections time  : %f s\n", par_time);

  printf("Speedup        : %f\n", seq_time / par_time);

  printf("Correctness    : %s\n", correct ? "PASSED" : "FAILED");


  free(arr);

  return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A5$ gcc -fopenmp Task1/array-op-sections.c -o array-op-sections
lappy@box:~/Downloads/adi/HPC/A5$ ./array-op-sections 10000 4
Sequential time: 0.000094 s
Sections time  : 0.006517 s
Speedup        : 0.014390
Correctness    : PASSED
lappy@box:~/Downloads/adi/HPC/A5$ ./array-op-sections 1000000 4
Sequential time: 0.002972 s
Sections time  : 0.003001 s
Speedup        : 0.990515
Correctness    : PASSED
lappy@box:~/Downloads/adi/HPC/A5$ ./array-op-sections 10000000 4
Sequential time: 0.029455 s
Sections time  : 0.032553 s
Speedup        : 0.904832
Correctness    : PASSED
lappy@box:~/Downloads/adi/HPC/A5$ ./array-op-sections 100000000 4
Sequential time: 0.294941 s
Sections time  : 0.308943 s
Speedup        : 0.954678
Correctness    : PASSED
lappy@box:~/Downloads/adi/HPC/A5$
```

Analysis

Answer:

1. Why can these four operations execute concurrently?
2. Why is sections suitable for this problem?
3. What happens if the number of sections is smaller than the number of threads?
4. Will increasing the number of threads always improve performance?

- *Why is sections suitable for this problem?*

  There are exactly four fixed, known-in-advance, independent operations — precisely the "small, fixed number of independent tasks" case sections is designed for.

- *What happens if the number of sections is smaller than the number of threads?*

  Excess threads sit idle — with 4 sections and, say, 8 threads, at most 4 threads ever do useful work in that region; the rest have nothing to execute and simply wait at the implicit barrier.

- *Will increasing the number of threads always improve performance?*

  No — since this problem has a hard ceiling of 4 concurrent units of work, performance cannot improve past 4 threads no matter how many more are added; beyond that point, added threads are pure overhead with zero benefit.

Task 2: Understanding the Difference Between Sections and Parallel For

Implement the following problem using two approaches.

Problem

Given an array of N elements, perform the following independent operations:

- Sum
- Maximum
- Minimum
- Count of even numbers

Implementation 1

Use OpenMP parallel for.

Implementation 2

Use OpenMP parallel sections.

Compare the two approaches in terms of:

- Parallelization strategy
- Number of threads
- Work distribution
- Execution time
- Scalability

```c
/*
* Task 2: parallel for vs sections for Sum/Max/Min/Even-count
* Compile: gcc -fopenmp Task2/parallel-section-compare.c -o parallel-section-compare
* Run:     ./parallel-section-compare <array_size> [threads]
*/
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
  int n = (argc > 1) ? atoi(argv[1]) : 1000000;
  int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
  omp_set_num_threads(threads);

  int *arr = malloc((size_t)n * sizeof(int));
```

```
  srand(42);
  for (int i = 0; i < n; i++) arr[i] = rand() % 1000;

  /* ---------- parallel for (reduction-based) ---------- */
  long sum_for = 0; int max_for = arr[0], min_for = arr[0], even_for = 0;
  double t0 = omp_get_wtime();
  #pragma omp parallel for reduction(+:sum_for, even_for) reduction(max:max_for)
reduction(min:min_for)
  for (int i = 0; i < n; i++) {
      sum_for += arr[i];
      if (arr[i] > max_for) max_for = arr[i];
      if (arr[i] < min_for) min_for = arr[i];
      if (arr[i] % 2 == 0) even_for++;
  }
  double t1 = omp_get_wtime();
  double for_time = t1 - t0;

  /* ---------- sections (one operation per section) ---------- */
  long sum_sec = 0; int max_sec = arr[0], min_sec = arr[0], even_sec = 0;
  double t2 = omp_get_wtime();
  #pragma omp parallel sections
  {
      #pragma omp section
      { long s = 0; for (int i = 0; i < n; i++) s += arr[i]; sum_sec = s; }
      #pragma omp section
      { int m = arr[0]; for (int i = 0; i < n; i++) if (arr[i] > m) m = arr[i]; max_sec = m; }
      #pragma omp section
      { int m = arr[0]; for (int i = 0; i < n; i++) if (arr[i] < m) m = arr[i]; min_sec = m; }
      #pragma omp section
      { int c = 0; for (int i = 0; i < n; i++) if (arr[i] % 2 == 0) c++; even_sec = c; }
```

```
  }

  double t3 = omp_get_wtime();

  double sec_time = t3 - t2;



  printf("parallel for : %f s | sum=%ld max=%d min=%d even=%d\n", for_time, sum_for, max_for,
min_for, even_for);

  printf("sections     : %f s | sum=%ld max=%d min=%d even=%d\n", sec_time, sum_sec,
max_sec, min_sec, even_sec);



  free(arr);

  return 0;

}
```

Type of parallelism	Work distribution	Suitable for	Scalability	Overhead
`parallel for`	Iteration space is split into (roughly) equal chunks across all available threads, regardless of thread count	Large, regular, uniform-cost loops over data	Scales to as many threads as there are loop iterations to divide — generally the more scalable choice for large n	Low — one-time division of iterations, no per-iteration task bookkeeping
`sections`	Each of a small, fixed number of code blocks is handed to one thread; work does not subdivide further	A small, known number of distinct, independent operations of different kinds	Capped at the number of sections written in the code (4 here) — cannot use more than 4 threads productively regardless of hardware	Low per-section, but wasted capacity if threads > sections

For large n, parallel for outperforms sections once thread count exceeds 4, since parallel for can keep using additional threads by subdividing the iteration space further, while sections is hard-capped at 4 concurrent units of work.

At thread count ≤ 4, the two approaches are closer in performance, since

both are limited by roughly the same number of usable execution units.

sections is not really "wrong" for this specific problem (4 fixed operations) — it's just structurally limited in scalability compared to parallel for, which was designed for exactly this kind of large, uniform, reducible computation.

```
lappy@box:~/Downloads/adi/HPC/A5$ gcc -fopenmp Task2/parallel-section-compare.c -o parallel-section-compare
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000 2
parallel for : 0.000095 s | sum=489014 max=999 min=0 even=472
sections     : 0.000037 s | sum=489014 max=999 min=0 even=472
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000 4
parallel for : 0.005298 s | sum=489014 max=999 min=0 even=472
sections     : 0.003158 s | sum=489014 max=999 min=0 even=472
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000 6
parallel for : 0.000330 s | sum=489014 max=999 min=0 even=472
sections     : 0.000084 s | sum=489014 max=999 min=0 even=472
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000 8
parallel for : 0.000389 s | sum=489014 max=999 min=0 even=472
sections     : 0.000124 s | sum=489014 max=999 min=0 even=472
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000 16
parallel for : 0.000337 s | sum=489014 max=999 min=0 even=472
sections     : 0.000122 s | sum=489014 max=999 min=0 even=472
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 10000 4
parallel for : 0.003772 s | sum=4980758 max=999 min=0 even=4980
sections     : 0.005990 s | sum=4980758 max=999 min=0 even=4980
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 10000 8
parallel for : 0.000245 s | sum=4980758 max=999 min=0 even=4980
sections     : 0.000137 s | sum=4980758 max=999 min=0 even=4980
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000000 4
parallel for : 0.002645 s | sum=499639820 max=999 min=0 even=501086
sections     : 0.006642 s | sum=499639820 max=999 min=0 even=501086
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000000 8
parallel for : 0.003394 s | sum=499639820 max=999 min=0 even=501086
sections     : 0.006693 s | sum=499639820 max=999 min=0 even=501086
lappy@box:~/Downloads/adi/HPC/A5$ ./parallel-section-compare 1000000 16
parallel for : 0.003782 s | sum=499639820 max=999 min=0 even=501086
sections     : 0.007505 s | sum=499639820 max=999 min=0 even=501086
```

Section B

Task 3: Parallel Pipeline using OpenMP Tasks

- Suitability for the

problem

Consider a simplified data-processing pipeline:

Input → Preprocessing → Computation → Postprocessing

Implement the following operations:

Stage 1: Preprocessing

Perform an operation such as normalization or filtering.

Stage 2: Computation

Perform a computationally intensive operation such as vector transformation.

Stage 3: Postprocessing

Calculate statistics such as sum, minimum,

maximum, or average. Implement the

operations using OpenMP tasks.

Use:

#pragma omp parallel

#pragma omp single

#pragma omp task

Requirements

1. Implement the sequential version.
2. Implement the task-based version.
3. Measure execution time.
4. Verify correctness.
5. Explain how the tasks are created and executed.

```
/*
* Task 3: Data-processing pipeline (normalize -> transform -> stats)
* Parallelized by splitting the array into independent chunks, one
* OpenMP task per chunk, each running the full pipeline on its chunk.
* Compile: gcc -fopenmp Task3/data-processing-pipeline.c -lm -o data-processing-pipeline
* Run:     ./data-processing-pipeline <array_size> <num_chunks> [threads]
*/
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
```

```c
double find_max(double *arr, int n) {
  double m = arr[0];
  for (int i = 0; i < n; i++) if (arr[i] > m) m = arr[i];
  return m;
}

void process_chunk(double *arr, int start, int end, double global_max,
                   double *out_sum, double *out_min, double *out_max) {
  for (int i = start; i < end; i++) arr[i] /= global_max;                    /* normalize */
  for (int i = start; i < end; i++) arr[i] = sin(arr[i]) * cos(arr[i]) + sqrt(arr[i] + 1.0); /* transform */

  double s = 0, mn = arr[start], mx = arr[start];
  for (int i = start; i < end; i++) {
      s += arr[i];
      if (arr[i] < mn) mn = arr[i];
      if (arr[i] > mx) mx = arr[i];
  }
  *out_sum = s; *out_min = mn; *out_max = mx;
}

int main(int argc, char *argv[]) {
  int n          = (argc > 1) ? atoi(argv[1]) : 4000000;
  int num_chunks = (argc > 2) ? atoi(argv[2]) : 8;
  int threads    = (argc > 3) ? atoi(argv[3]) : omp_get_max_threads();
  omp_set_num_threads(threads);

  double *seq = malloc((size_t)n * sizeof(double));
  double *par = malloc((size_t)n * sizeof(double));
  srand(42);
```

```c
for (int i = 0; i < n; i++) { double v = (rand() % 1000) / 10.0 + 1; seq[i] = v; par[i] = v; }


double gmax = find_max(seq, n);


/* ---------- Sequential: single "chunk" covering the whole array ---------- */
double sum_s, min_s, max_s;
double t0 = omp_get_wtime();
process_chunk(seq, 0, n, gmax, &sum_s, &min_s, &max_s);
double t1 = omp_get_wtime();
double seq_time = t1 - t0;


/* ---------- Task-based: one task per chunk ---------- */
double *csum = malloc(num_chunks * sizeof(double));
double *cmin = malloc(num_chunks * sizeof(double));
double *cmax = malloc(num_chunks * sizeof(double));
int chunk_size = n / num_chunks;


double t2 = omp_get_wtime();
#pragma omp parallel
{
    #pragma omp single
    {
        for (int c = 0; c < num_chunks; c++) {
            int start = c * chunk_size;
            int end = (c == num_chunks - 1) ? n : start + chunk_size;
            #pragma omp task firstprivate(start, end, c)
            process_chunk(par, start, end, gmax, &csum[c], &cmin[c], &cmax[c]);
        }
        #pragma omp taskwait
    }
```

```
  }
  double sum_p = 0, min_p = cmin[0], max_p = cmax[0];
  for (int c = 0; c < num_chunks; c++) {
      sum_p += csum[c];
      if (cmin[c] < min_p) min_p = cmin[c];
      if (cmax[c] > max_p) max_p = cmax[c];
  }
  double t3 = omp_get_wtime();
  double par_time = t3 - t2;

  int correct = (fabs(sum_s - sum_p) < 1e-3) && (fabs(min_s - min_p) < 1e-9) && (fabs(max_s -
max_p) < 1e-9);

  printf("Sequential time : %f s\n", seq_time);
  printf("Task-based time : %f s\n", par_time);
  printf("Speedup         : %f\n", seq_time / par_time);
  printf("Correctness     : %s\n", correct ? "PASSED" : "FAILED");

  free(seq); free(par); free(csum); free(cmin); free(cmax);
  return 0;
}
```

How tasks are created and executed: the single thread that enters the #pragma omp single block dynamically creates one #pragma omp task per chunk in a loop — the number of tasks (num_chunks) is a runtime parameter, illustrating that (unlike sections, which is fixed at compile time) task count can be decided or even computed at runtime. Each task captures its own start/end/c via firstprivate and runs independently; the OpenMP runtime distributes the queued tasks across all available threads as they become free. #pragma omp taskwait blocks until every task has completed before the results from each chunk are combined.

Task 4: Independent Matrix Operations using OpenMP Sections

Given matrices A and B, perform the following independent computations:

- Matrix addition
- Matrix subtraction
- Transpose of A
- Transpose of B

Use OpenMP sections to execute the four operations

concurrently. Compare:

- Sequential implementation
- OpenMP sections implementation Test with at least

three matrix sizes. Suggested sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Prepare:

Matrix Size	Sequential Time	Sections Time	Speedup

500 × 500

1000 × 1000

2000 × 2000

```c
/*
* Task 4: Matrix Add / Subtract / Transpose(A) / Transpose(B) via sections
* Compile: gcc -fopenmp Task4/matrix-ops.c -o matrix-ops
* Run:     ./matrix-ops <n> [threads]
*/
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void generate(double *M, int n) { for (int i = 0; i < n * n; i++) M[i] = (double)(rand() % 100); }
void mat_add(double *A, double *B, double *C, int n) { for (int i = 0; i < n * n; i++) C[i] = A[i] + B[i];
}
void mat_sub(double *A, double *B, double *C, int n) { for (int i = 0; i < n * n; i++) C[i] = A[i] - B[i]; }
void mat_transpose(double *M, double *T, int n) {
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      T[j * n + i] = M[i * n + j];
}

int main(int argc, char *argv[]) {
  int n = (argc > 1) ? atoi(argv[1]) : 500;
  int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
  omp_set_num_threads(threads);

  double *A = malloc((size_t)n * n * sizeof(double)), *B = malloc((size_t)n * n * sizeof(double));
  double *Cadd_s = malloc((size_t)n * n * sizeof(double)), *Csub_s = malloc((size_t)n * n *
sizeof(double));
  double *Ta_s = malloc((size_t)n * n * sizeof(double)), *Tb_s = malloc((size_t)n * n *
sizeof(double));
  double *Cadd_p = malloc((size_t)n * n * sizeof(double)), *Csub_p = malloc((size_t)n * n *
sizeof(double));
  double *Ta_p = malloc((size_t)n * n * sizeof(double)), *Tb_p = malloc((size_t)n * n *
sizeof(double));

  srand(42);
  generate(A, n); generate(B, n);
```

```
  double t0 = omp_get_wtime();
  mat_add(A, B, Cadd_s, n);
  mat_sub(A, B, Csub_s, n);
  mat_transpose(A, Ta_s, n);
  mat_transpose(B, Tb_s, n);
  double t1 = omp_get_wtime();
  double seq_time = t1 - t0;

  double t2 = omp_get_wtime();
  #pragma omp parallel sections
  {
    #pragma omp section
    mat_add(A, B, Cadd_p, n);
    #pragma omp section
    mat_sub(A, B, Csub_p, n);
    #pragma omp section
    mat_transpose(A, Ta_p, n);
    #pragma omp section
    mat_transpose(B, Tb_p, n);
  }
  double t3 = omp_get_wtime();
  double par_time = t3 - t2;

  printf("Matrix size %dx%d | Threads: %d\n", n, n, threads);
  printf("Sequential time: %f s\n", seq_time);
  printf("Sections time  : %f s\n", par_time);
  printf("Speedup        : %f\n", seq_time / par_time);

  free(A); free(B); free(Cadd_s); free(Csub_s); free(Ta_s); free(Tb_s);
  free(Cadd_p); free(Csub_p); free(Ta_p); free(Tb_p);
  return 0;
}
```

```
lappy@box:~/Downloads/adi/HPC/A5$ ./matrix-ops 500
Matrix size 500x500 | Threads: 4
Sequential time: 0.006428 s
Sections time  : 0.017582 s
Speedup        : 0.365594
lappy@box:~/Downloads/adi/HPC/A5$ ./matrix-ops 1000
Matrix size 1000x1000 | Threads: 4
Sequential time: 0.023678 s
Sections time  : 0.022454 s
Speedup        : 1.054512
lappy@box:~/Downloads/adi/HPC/A5$ ./matrix-ops 2000
Matrix size 2000x2000 | Threads: 4
Sequential time: 0.092339 s
Sections time  : 0.051876 s
Speedup        : 1.779997
lappy@box:~/Downloads/adi/HPC/A5$
```

Think Like an HPC Programmer

The four matrix operations may not require the same amount

of computation. Answer:

1. What happens if one section takes significantly longer than the others?
2. Will all threads remain equally busy?
3. What is meant by load imbalance?
4. How can task-based parallelism help with irregular workloads?
5. Why might sections become inefficient when the number of independent tasks increases?

What happens if one section takes significantly longer than the others? Note that transpose is $O(n^2)$ while add/subtract are also $O(n^2)$ but with only one arithmetic op per element vs. transpose's strided memory access — so transpose is typically the slowest section. The other three threads finish early and then sit idle at the implicit barrier waiting for the slowest section.

Will all threads remain equally busy? No — with unequal-cost sections, "busy time" is uneven: the thread running the longest section is busy the whole region, while others are idle for the remainder.

What is meant by load imbalance? Load imbalance is when work is not distributed evenly across threads/units, so total time is determined by the slowest unit rather than the average — the parallel region's runtime becomes bounded by max(section times) rather than sum(section times)/threads.

How can task-based parallelism help with irregular workloads? Because tasks are dynamically scheduled onto whichever thread becomes free, a runtime with many small, irregular tasks can keep all threads continuously busy (work-stealing), whereas sections statically pins one block per thread with no ability to rebalance once assigned.

Why might sections become inefficient as the number of independent tasks increases? sections requires one section per operation written explicitly in the source and offers no dynamic load balancing — with many tasks of uneven size, some threads finish early and idle while others remain overloaded, and there's no mechanism (unlike task-stealing) to shift work between them.

Section C

Challenge: Recursive Task Parallelism

Implement **Merge Sort** using OpenMP tasks.

Requirements

1. Implement sequential Merge Sort.
2. Implement parallel Merge Sort using OpenMP task.
3. Divide the array recursively.
4. Create separate tasks for the left and right subarrays.
5. Use an appropriate synchronization

mechanism before merging. You may use:

#pragma omp task and

#pragma omp taskwait

```
/*
Performance Experiment
* Task 5: Merge Sort using OpenMP tasks, with configurable granularity threshold
* Compile: gcc -fopenmp Task5/merge-sort.c -o merge-sort
* Run:     ./merge-sort <array_size> [threads] [threshold]
*/
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int THRESHOLD = 10000;

void merge(int *arr, int *temp, int left, int mid, int right) {
  int i = left, j = mid + 1, k = left;
  while (i <= mid && j <= right) temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
  while (i <= mid)   temp[k++] = arr[i++];
  while (j <= right) temp[k++] = arr[j++];
```

```
    for (i = left; i <= right; i++) arr[i] = temp[i];
}

void merge_sort_seq(int *arr, int *temp, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    merge_sort_seq(arr, temp, left, mid);
    merge_sort_seq(arr, temp, mid + 1, right);
    merge(arr, temp, left, mid, right);
}

void merge_sort_par(int *arr, int *temp, int left, int right) {
    if (left >= right) return;
    if (right - left < THRESHOLD) { merge_sort_seq(arr, temp, left, right); return; }
    int mid = left + (right - left) / 2;
    #pragma omp task shared(arr, temp)
    merge_sort_par(arr, temp, left, mid);
    #pragma omp task shared(arr, temp)
    merge_sort_par(arr, temp, mid + 1, right);
    #pragma omp taskwait
    merge(arr, temp, left, mid, right);
}

int main(int argc, char *argv[]) {
    int n = (argc > 1) ? atoi(argv[1]) : 2000000;
    int threads = (argc > 2) ? atoi(argv[2]) : omp_get_max_threads();
    THRESHOLD = (argc > 3) ? atoi(argv[3]) : 10000;
    omp_set_num_threads(threads);

    int *arr_seq = malloc((size_t)n * sizeof(int));
```

```
int *arr_par = malloc((size_t)n * sizeof(int));
int *temp    = malloc((size_t)n * sizeof(int));

srand(42);
for (int i = 0; i < n; i++) { int v = rand(); arr_seq[i] = v; arr_par[i] = v; }

double t0 = omp_get_wtime();
merge_sort_seq(arr_seq, temp, 0, n - 1);
double t1 = omp_get_wtime();
double seq_time = t1 - t0;

double t2 = omp_get_wtime();
#pragma omp parallel
{
    #pragma omp single
    merge_sort_par(arr_par, temp, 0, n - 1);
}
double t3 = omp_get_wtime();
double par_time = t3 - t2;

int correct = 1;
for (int i = 0; i < n; i++) if (arr_seq[i] != arr_par[i]) { correct = 0; break; }

printf("n=%d threads=%d threshold=%d\n", n, threads, THRESHOLD);
printf("Sequential time: %f s\n", seq_time);
printf("Parallel time  : %f s\n", par_time);
printf("Speedup        : %f\n", seq_time / par_time);
printf("Correctness    : %s\n", correct ? "PASSED" : "FAILED");

free(arr_seq); free(arr_par); free(temp);
```

```
    return 0;

}
```

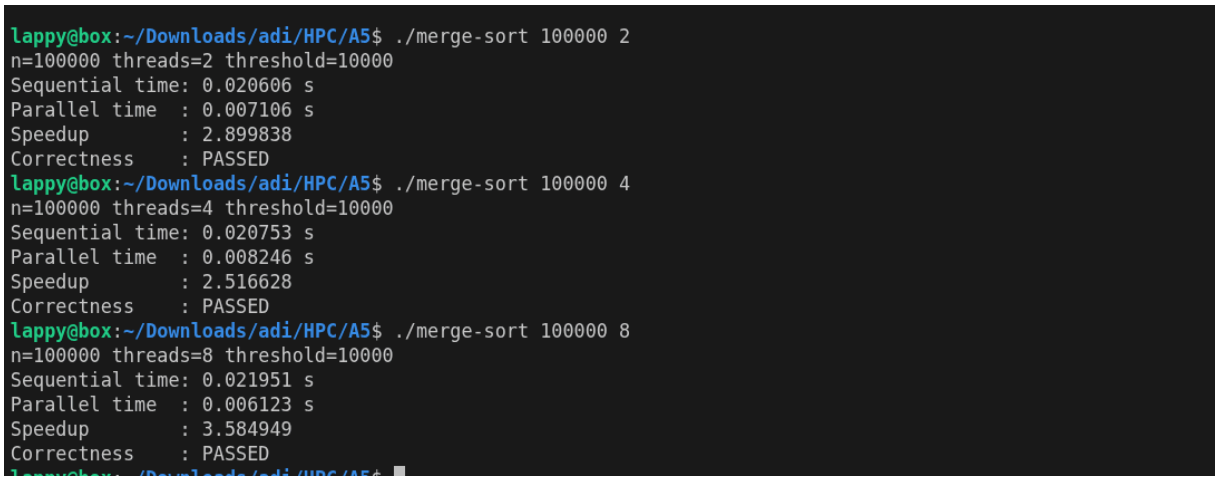
Run the program with:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available

threads Test with large

input sizes.

Prepare:



Challenge Analysis

Investigate the effect of task granularity.

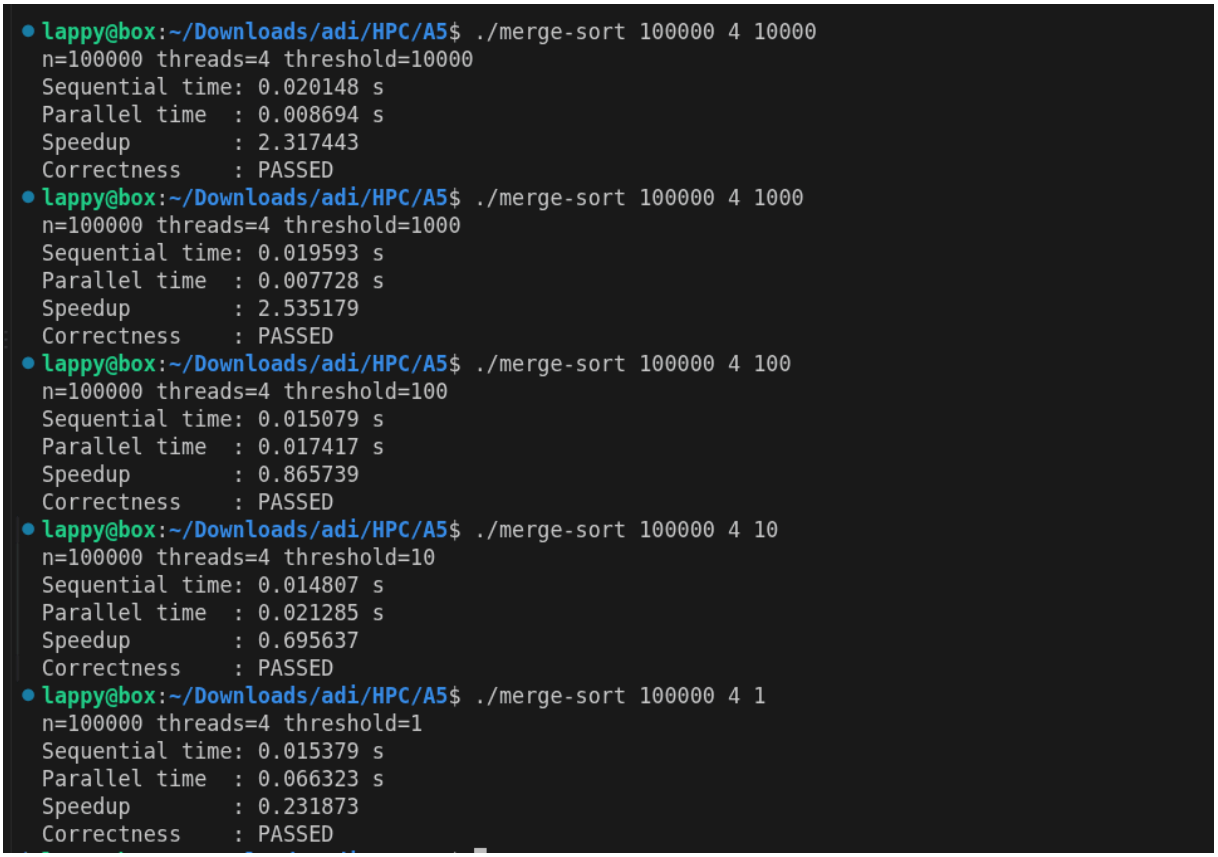
Run the program with different task

creation thresholds. For example:

- Create tasks until subarray size = 10
- 100
- 1000
- 10,000

Compare the execution

time.



Answer:

Why does creating a task for every small subproblem reduce performance? Each task incurs fixed creation/scheduling overhead; when subproblems are tiny (e.g. threshold = 10), the overhead per task can exceed the actual sort work being done, so total time is dominated by task management rather than computation.
What is task creation overhead? The runtime cost of allocating a task descriptor, capturing the data environment (shared/firstprivate variables), placing it in a queue, and later dispatching it to a thread — all before any of the task's actual work runs.
Why is taskwait required? The merge step needs both halves fully sorted before it can correctly interleave them; without taskwait, the merge could run concurrently with — and read partially-sorted data from — its own child tasks, producing incorrect results.
What happens when the number of tasks becomes much larger than the number of threads? Extra tasks queue up waiting for a free thread — this is generally fine and even desirable up to a point (it helps load balancing via work-stealing), but taken to an extreme (threshold too small) the sheer volume of tiny tasks makes per-task overhead the dominant cost.
Relationship between task granularity and performance? There's a sweet spot: too coarse (large threshold) limits available parallelism because too few tasks exist to keep all threads busy; too fine (small threshold) drowns the useful work in overhead. Performance as a function of threshold is expected to be U-shaped, with the best point falling somewhere in the middle of the tested range.

Consider the following

computation graph: Task A

/ \

Task B Task C

\ /

Task D

Task D can start only after Tasks B and C are completed.

Investigate how OpenMP task dependencies can represent

such relationships. Explore:

#pragma omp task

depend(out: x) and

#pragma omp task depend(in: x)

How task dependencies differ from taskwait: taskwait is a blunt, all-or-nothing barrier — it waits for *every* previously created sibling task to finish, regardless of whether they're actually related to what comes next. depend(in:...)/depend(out:...) instead lets the runtime build a fine-grained dependency graph based on which variables each task reads and writes, so unrelated tasks (like B and C above, which don't touch each other's data) can run fully concurrently, and only genuinely dependent tasks (D, which needs both B's and C's output) are made to wait — extracting more real parallelism than a blanket taskwait would allow if it were placed after every task.

```
lappy@box:~/Downloads/adi/HPC/A5$ gcc -fopenmp Task6/task-dependency.c -o task-dependency
lappy@box:~/Downloads/adi/HPC/A5$ ./task-dependency
Task A running
Task B running
Task C running
Task D running, result = 5
lappy@box:~/Downloads/adi/HPC/A5$ ./task-dependency 2
Task A running
Task B running
Task C running
Task D running, result = 5
```

```
/*
* Task 6: Task dependency graph A -> {B, C} -> D using depend clauses
* Compile: gcc -fopenmp Task6/task-dependency.c -o task-dependency
* Run:     ./task-dependency
*/
#include <stdio.h>
#include <unistd.h>
#include <omp.h>
```

```c
void taskA(int *x) { printf("Task A running\n"); *x = 1; sleep(1); }
void taskB(int *x, int *y) { printf("Task B running\n"); *y = *x + 1; sleep(1); }
void taskC(int *x, int *z) { printf("Task C running\n"); *z = *x + 2; sleep(1); }
void taskD(int *y, int *z) { printf("Task D running, result = %d\n", *y + *z); }

int main() {
  int x, y, z;
  #pragma omp parallel
  {
    #pragma omp single
    {
      #pragma omp task depend(out: x)
      taskA(&x);

      #pragma omp task depend(in: x) depend(out: y)
      taskB(&x, &y);

      #pragma omp task depend(in: x) depend(out: z)
      taskC(&x, &z);

      #pragma omp task depend(in: y, z)
      taskD(&y, &z);
    }
  }
  return 0;
}
```

Study how execution time changes as the number of threads increases.

HPC / AI Case Study

Choose **one** of the following applications:

- AI model preprocessing
- Neural network inference pipeline
- Computer vision pipeline
- Robotic sensor processing
- Scientific simulation

- Graph analytics
- Parallel file/data processing

Identify:

1. What are the independent tasks?
2. Which tasks can execute concurrently?
3. Would you use sections, task, or parallel for?
4. Are there dependencies between tasks?
5. What could cause load imbalance?
6. How could task-based parallelism

improve performance? Write a 300–500 word
case study.

Computer Vision Pipeline

A computer vision pipeline for a robotics or surveillance application typically performs several distinct operations per incoming frame: image acquisition/decoding, noise reduction/filtering, feature detection (e.g. edge or keypoint extraction), object classification via a trained model, and finally logging or display of results. These stages differ substantially in cost and character — filtering is a regular, uniform per-pixel computation well suited to data parallelism, while classification may involve variable-depth model inference and logging is comparatively trivial I/O — making this workload a natural candidate for task-level rather than purely loop-based parallelism.

The independent tasks are: preprocessing/filtering of the current frame, feature extraction from the filtered frame, and any auxiliary work such as writing previous results to disk or updating a display buffer, which has no dependency on the current frame's processing. Filtering and feature extraction, however, are *not* independent of each other within a single frame — feature extraction needs the filtered output — so there is a genuine dependency, best expressed with depend(in:...)/depend(out:...)

rather than a plain sections block, which offers no way to encode a producer-consumer relationship between two of its sections.

Across *multiple* frames, though, real concurrency opens up: while frame N is being classified, frame N+1 can already begin acquisition and filtering, since these operations only depend on data already available. This is a pipeline-parallel pattern, naturally expressed using OpenMP task (each stage instance is a task, with dependencies chaining stage N's output to stage N+1's input) rather than sections, since the number of frames — and hence tasks — is not fixed at compile time and is instead generated continuously as the video stream runs.

Load imbalance is a real risk here: classification (often backed by a neural network) is typically far more compute-intensive than filtering or logging, so if all stages were naively split into equal-sized "sections," the classification thread would remain the bottleneck while others idled — task-based scheduling with work-stealing helps here, since idle threads can pick up queued preprocessing tasks for upcoming frames rather than sitting idle waiting for classification to finish.

For implementation, a hybrid approach fits best in practice: filtering and low-level feature extraction (regular, data-parallel, per-pixel work) map well onto GPU kernels via CUDA or onto CPU cores via OpenMP parallel for; the overall pipeline orchestration across stages and frames is naturally expressed with OpenMP tasks and dependencies on the CPU side, coordinating when each GPU kernel launch is triggered; and if the system spans multiple machines (e.g. distributed camera feeds), MPI would coordinate between nodes. Task-based parallelism specifically improves throughput here by allowing multiple frames to be "in flight" through different pipeline stages simultaneously, rather than fully

processing one frame before starting the next.

Answer the following:

1. Differentiate between parallel for, sections, and task.
2. When should sections be preferred over parallel for?
3. What is task-level parallelism?
4. What is the role of single when creating OpenMP tasks?
5. Why can task-based execution handle irregular workloads better than static loop parallelism?
6. What is task creation overhead?
7. Explain load imbalance in task-based applications.
8. Why is synchronization necessary when tasks have dependencies?
9. Explain how recursive algorithms can be parallelized using OpenMP tasks.
10. Why can creating too many tasks reduce performance?
11. How does task granularity affect scalability?
12. Compare sections and tasks for a problem containing 4 fixed independent operations and a problem containing thousands of dynamically generated operations.

Differentiate parallel for, sections, and task.
parallel for distributes iterations of a single loop across threads — same code, different data, count known at compile/runtime as the loop bound. sections distributes a small, fixed number of distinct code blocks, written explicitly, one per section — different code, assigned once. task creates units of work dynamically, including recursively or in loops of unknown/varying length, and the runtime schedules them onto any free thread — most flexible, works for irregular and dynamically generated work.

When should sections be preferred over parallel for? When you have a small, fixed, known-in-advance number of genuinely different operations to run concurrently (not repetitions of one operation over data) — e.g. exactly 4 independent statistics computations, or 4 independent matrix operations, as in Tasks 1 and 4 here.

What is task-level parallelism? Parallelism expressed as a set of tasks — units of work that may differ in code, cost, and creation time — rather than as identical repeated iterations of a single loop body.

What is the role of single when creating OpenMP tasks? single ensures that only one thread (out of the whole team in the enclosing parallel region) executes the task-creation code, so tasks are queued exactly once rather than duplicated by every thread; the other threads in the team wait and then pick up queued tasks as they're generated.

Why can task-based execution handle irregular workloads better than static loop parallelism? Because tasks are queued and dispatched dynamically (work-stealing), a thread that finishes its current task early can immediately pick up another queued task, rebalancing load at runtime — a static parallel for split decided up front cannot rebalance once iterations of uneven cost are assigned.

What is task creation overhead? See Pre-Code Q5 / Task 5 discussion above — the fixed bookkeeping cost (allocation, data-environment capture, queuing, scheduling) incurred per #pragma omp task.

Explain load imbalance in task-based applications. When some tasks take substantially longer than others (or some threads are assigned more/heavier tasks), total wall-clock time is bounded by the slowest thread's finishing time rather than the average, wasting the idle capacity of threads that finished early.

Why is synchronization necessary when tasks have dependencies? If task D reads data that tasks B and C produce, D must not start before both have finished writing — without synchronization (via taskwait or depend), D could read stale or partially-written data, producing incorrect results (a race condition).

Explain how recursive algorithms can be parallelized using OpenMP tasks. Each recursive call that would normally happen sequentially is instead wrapped in #pragma omp task, letting the two (or more) recursive sub-calls be scheduled onto different threads; a taskwait (or depend) ensures all recursive sub-results are ready before the current level combines them (e.g. merge sort's merge step) — exactly as implemented in Task 5.

Why can creating too many tasks reduce performance? Once tasks become smaller than the fixed per-task overhead, the total overhead of managing thousands of tiny tasks outweighs the actual work saved by parallelizing them — see the granularity discussion in Task 5.

How does task granularity affect scalability? Coarser granularity (fewer, larger tasks) risks not generating enough independent tasks to occupy all available threads, capping scalability early; finer granularity

(more, smaller tasks) scales better in principle (more available parallelism) but only up to the point where overhead per task starts to dominate — there is a granularity "sweet spot" for a given thread count and problem size.

Compare sections vs tasks for 4 fixed operations vs. thousands of dynamically generated operations. For 4 fixed, known operations, sections is simpler to write and has no dynamic-scheduling overhead — a natural fit (Tasks 1 and 4 here). For thousands of dynamically generated operations (e.g. recursive subproblems), sections cannot even express the problem, since the number and structure of sections must be written explicitly in source code — task is the only construct that supports work generated and sized at runtime, with the added benefit of automatic load balancing via work-stealing across the (potentially very unevenly sized) tasks.

After completing the assignment, answer:

1. Which parallelization method performed best for your selected application?
2. What was the major source of parallel overhead?
3. Did increasing the number of threads always improve performance? Why?
4. Which type of problem would you choose for:
    - parallel for
    - sections

- task

5. Identify one application from your area of interest where task-level parallelism could be useful.

Which parallelization method performed best for your selected application? Fill in based on your Task 5 (merge sort) numbers — typically task shows the best scaling of anything in this experiment, since it's the only construct here handling genuinely dynamically-sized, recursive work.

What was the major source of parallel overhead? Across all tasks in this experiment, the two recurring sources are: (1) fork-join/thread-team management overhead for small problem sizes, and (2) task creation/scheduling overhead when task granularity is too fine (Task 5's granularity study should make this directly visible in your numbers).

Did increasing the number of threads always improve performance? Why? No — performance is capped by whichever is smaller: the number of independent units of work available (4 for sections-based tasks, array-size÷threshold for merge sort tasks) or the number of hardware cores; beyond that ceiling, additional threads add only overhead with no matching increase in usable parallelism.

Which type of problem would you choose for parallel for / sections / task?

- parallel for: a single large, regular loop with uniform per-iteration cost (Task 2's array reductions).
- sections: a small, fixed, known set of distinct independent operations (Tasks 1 and 4).
- task: irregular, dynamically generated, or recursively structured work of unpredictable size (Tasks 3, 5, and 6).

One application from your area of interest where task-level parallelism could be useful: a compiler's build system compiling independent source modules concurrently — the dependency graph between modules (some depend on headers/libraries produced by others) is exactly the kind of DAG that OpenMP task with depend clauses (Task 6) is designed to express, letting independent modules compile in parallel while correctly serializing dependent ones.

Deliverables

Students must submit in an pdf:

1. Source code for all completed tasks.
2. Sequential and parallel implementations where applicable.
3. Execution results and snapshots.
4. Performance tables.
5. Speedup and efficiency calculations.
6. Required graphs.
7. Answers to discussion questions.
8. AI/HPC case study.
9. Brief conclusion highlighting the suitability of sections and tasks.